

Detector de sonrisas

Instalar OpenCV

```
pip install OpenCV-python
```

1. Importación de librerías

```
import cv2
```

¿Qué es esto?

- cv2 es la librería de OpenCV en Python.
- Proporciona herramientas para leer video, detectar objetos, dibujar sobre imágenes y más.

2. Cargar los clasificadores Haar

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +  
'haarcascade_frontalface_default.xml')
```

```
smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +  
'haarcascade_smile.xml')
```

¿Qué hacen?

- Los **clasificadores Haar** son modelos preentrenados para detectar patrones (caras, ojos, sonrisas).
- OpenCV ya trae muchos de estos XML entrenados.
- Aquí cargamos dos:
 - Uno para **detectar rostros**.
 - Otro para **detectar sonrisas** dentro del rostro.

Se basan en "patrones de contraste" comunes en caras humanas (ej. ojos oscuros vs frente clara).

3. Activar la cámara web

```
cap = cv2.VideoCapture(0)
```

¿Qué significa?

- VideoCapture(0) activa la **cámara predeterminada** del equipo.
- Si hay más cámaras conectadas, puedes probar con 1, 2, etc.

4. Bucle de lectura de video

```
while True:
```

```
ret, frame = cap.read()
```

```
if not ret:
```

```
    break
```

¿Qué hace?

- Lee cada **fotograma del video** (como una imagen).
- ret es un valor booleano: True si se leyó correctamente.
- frame contiene la imagen actual.

5. Preprocesamiento (conversión a escala de grises)

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

¿Por qué convertir a gris?

- La detección de rostros y sonrisas **funciona mejor y más rápido en blanco y negro**.
- El modelo Haar fue entrenado sobre imágenes grises, no en color.

6. Detección de rostros

```
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)
```

¿Qué hace esto?

- Busca **rostros** en la imagen gris.
- detectMultiScale devuelve una lista de rectángulos (x, y, ancho, alto) donde encontró caras.
- Parámetros:
 - scaleFactor: cómo reduce la imagen en cada paso (más bajo = más preciso, pero más lento).
 - minNeighbors: cuántos "rectángulos vecinos" necesita para aceptar un rostro.

En este caso: scaleFactor=1.3

Indica **cuánto se reduce la imagen** en cada paso para detectar objetos a diferentes tamaños.

- **Valores comunes:** 1.1 (más preciso), 1.3 (más rápido pero menos preciso).
- **Ejemplo:** Si es 1.3, la imagen se reduce a 130%, 169%, etc., en cada nivel de escala.

Cuanto menor el valor, más escalas revisa → más lento pero más preciso.

minNeighbors=5

Este parámetro indica **cuántas detecciones vecinas** necesita un rectángulo para ser considerado una detección válida.

- **Más alto = más exigente (menos falsos positivos)**
- **Más bajo = más sensible (pero puede detectar cosas erróneas)**

minNeighbors=5 es un buen equilibrio.

Si minNeighbors=1: acepta casi todo → **muy sensible** → puede haber errores o falsos positivos.

Si minNeighbors=5: solo acepta si hay al menos **5 coincidencias cercanas** → más confiable.

Si minNeighbors=10: solo acepta si está **muy seguro** → puede perder caras reales.

7. Por cada rostro, buscar sonrisa

for (x, y, w, h) in faces:

```
roi_gray = gray[y:y+h, x:x+w]
```

```
smiles = smile_cascade.detectMultiScale(roi_gray, scaleFactor=1.8, minNeighbors=20)
```

¿Qué hace?

- Toma una **región de interés (ROI)**: el rostro detectado.
- Dentro de ese rostro, busca si hay **una sonrisa**.
- Se usa otro detectMultiScale, esta vez con parámetros más exigentes (porque las sonrisas son más pequeñas y variables).

OpenCV nos devuelve una **lista de rectángulos**, donde cada rectángulo representa una **cara detectada**. Cada uno de esos rectángulos viene expresado en cuatro valores:

Variable	Significado
x	Posición horizontal (izquierda)
y	Posición vertical (superior)
w	Ancho del rectángulo (width)
h	Alto del rectángulo (height)

Así que (x, y) es la **esquina superior izquierda** del rostro, y w, h son el **tamaño** del rostro.

- gray: es la imagen completa en escala de grises.
- gray[y:y+h, x:x+w]: es una **sección (o recorte) de la imagen**.
- Esa sección corresponde a un **rostro detectado**.

ROI = Region Of Interest (Región de Interés)

Entonces, esa línea **extrae solo la parte de la imagen donde está la cara** para luego trabajar en ella, por ejemplo para buscar una sonrisa.

Ejemplo visual (imaginario)

Supón que la cara detectada está en estas coordenadas:

- $x = 100, y = 50, w = 80, h = 80$

Eso significa que hay una cara desde:

- Horizontal: de 100 a 180 ($100 + 80$)
- Vertical: de 50 a 130 ($50 + 80$)

Entonces:

```
roi_gray = gray[50:130, 100:180]
```

→ Se recorta esa zona y se guarda como roi_gray.

¿Por qué se usa?

Porque así analizamos **solo la cara**, y no toda la imagen. Esto:

- Aumenta la velocidad
- Mejora la precisión
- Evita detectar sonrisas fuera del rostro

8. Dibujar resultados en el video

```
if len(smiles) > 0: cv2.putText(frame, 'Sonriendo', (x, y - 10),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2) cv2.rectangle(frame, (x, y), (x+w,
y+h), (0, 255, 0), 2)
```

```
else:
```

```
cv2.putText(frame, 'Serio', (x, y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
```

¿Qué hace?

- Si se detectaron sonrisas:
 - Se escribe “Sonriendo” sobre la cara.
 - Se dibuja un **rectángulo verde**.
- Si no hay sonrisa:
 - Se escribe “Serio”.

- Se dibuja un **rectángulo rojo**.

9. Mostrar el video en vivo

```
cv2.imshow('Detector de Sonrisas', frame)
```

¿Qué hace?

- Abre una ventana con el nombre 'Detector de Sonrisas' y muestra el fotograma con las etiquetas.

10. Salir del programa

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

¿Qué hace?

- Espera 1 milisegundo y verifica si presionaste la tecla q.
- Si sí, sale del bucle while.

11. Liberar la cámara y cerrar ventanas

```
cap.release()
```

```
cv2.destroyAllWindows()
```

¿Qué hace?

- `release()`: libera la cámara.
- `destroyAllWindows()`: cierra todas las ventanas que cv2 haya creado.